

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

School of Information Technology
Department of Computer Science
COS326 – Database Systems
Practical 4 2026

Release Date:	05 October 2026
Submission Date and Time:	18 October 2026 @ 23:59Hrs
Lecturer:	Mr S.M Makura
Total: 50 Marks	

A. Objectives

1. Gain practical experience with the MongoDB document database management system.
2. Create and manipulate MongoDB databases, collections and documents.
3. Develop reusable JavaScript functions for the MongoDB shell.
4. Perform MongoDB insert, find, update and delete operations.
5. Work with arrays and embedded documents in MongoDB.
6. Use MongoDB aggregation pipelines to analyse security event data.
7. Perform text searches on MongoDB documents.
8. Develop a Java application that accesses and manipulates data stored in MongoDB.
9. Appreciate how MongoDB can be used to manage semi-structured cybersecurity data.

B. Submission Procedure

1. A file named **.mongoshrc.js** containing all JavaScript functions developed for Task 1 and 2.

2. Java application - submit the complete Java project containing all source code required to run the application developed in Task 3.

3. The completed **AI declaration form** where you detail the AI prompts you used and the usage types.

4. Compress the above documents into an archive and upload it to the ClickUP link for practical submissions before the due date/time. The file name for the

archive must have your student number as part of the file name, e.g. *uXXXXXXXX_Surname_Initials(XXXXXXXX is your student number) e.g u12345678_Smith_JN*

NO LATE submissions will be accepted after the submission date and time has lapsed. Do not wait until the last minute to submit and then provide reasons related to technical difficulties.

Important

- All queries must be written and tested using MongoDB
- Your queries must operate on the supplied json file and must not rely on manually entered results.

Academic Integrity: You may use AI-assisted tools (e.g., ChatGPT, Copilot, Gemini etc) to help you understand XQuery concepts and syntax. However, you remain responsible for the design, implementation, and correctness of your solution. During marking, you may be asked to explain or modify any part of your submission. Failure to demonstrate understanding may result in marks being deducted.

Question 1: CyberShield Security Operations Centre

Scenario

CyberShield Security Services (CSS) is a cybersecurity company that provides Security Operations Centre (SOC) services to organisations.

The CyberShield SOC receives security events from multiple sources, including:

- firewalls;
- intrusion detection systems;
- endpoint security systems;
- authentication servers; and
- network monitoring systems.

The SOC receives a large volume of security events every day. These events are not completely uniform. Different security devices may provide different information, and a single security event may contain arrays of tags, affected systems, and recommended response actions.

The CyberShield development team has decided to use MongoDB to store and analyse security events because its document-oriented model is suitable for handling this type of semi-structured security information.

You have been employed as a Junior Database Developer in the CyberShield SOC development team.

Your task is to develop a prototype MongoDB-based security event management system. You will create and manipulate security event documents, develop reusable MongoDB shell functions, perform queries and aggregation operations, and develop a Java application that allows SOC analysts to interact with the database.

Source: Makura S.M (2026)

TASK 1: MONGODB DATABASE AND SHELL FUNCTIONS [18 MARKS]

The CyberShield SOC requires several reusable functions to manage security events. For Task 1, use the **securityEvents** collection to test your functions. Before commencing Task 2, clear the collection and import the supplied `security_events.json` dataset again. Task 2 must be performed using the original supplied dataset. Open the `security_events.json` file provided for an idea of the document structure to insert.

Question 1.1 – Database and Collection [3 Marks]

Create the database **cybershield** and import the collection **securityEvents** which has been provided into MongoDB.

Question 1.2 - Insert Security Events [4 Marks]

Create a MongoDB shell function called:

```
insertSecurityEvents(dbName, colName, count)
```

The function must insert count security event documents into the **securityEvents** collection.

Each document must contain at least:

```
eventId
timestamp
sourceIP
destinationIP
eventType
severity
description
```

The function must generate a unique event ID for each inserted event. The function must display an appropriate message indicating how many documents were inserted.

Question 1.3 - Find Security Events [4 Marks]

Create a MongoDB shell function called:

```
findEventsByIP(dbName, colName, sourceIP)
```

The function must:

1. search the specified collection;
2. retrieve all security events generated by the supplied sourceIP; and
3. print the matching documents.

If no events are found, an appropriate message must be displayed.

Question 1.4 - Update Event Severity [4 Marks]

Create a MongoDB shell function called:

```
updateSeverity(dbName, colName, eventType, newSeverity)
```

The function must update the severity field for all security events matching the supplied eventType.

For example:

```
updateSeverity(
    "cybershield",
    "securityEvents",
    "Brute Force Attack",
    "Critical"
)
```

must change the severity of matching events to:

```
Critical
```

The function must display the number of documents modified.

Question 1.5 - Remove Low-Severity Events [3 Marks]

Create a MongoDB shell function called:

```
removeLowSeverityEvents(dbName, colName)
```

The function must remove all security events whose severity is:

```
Low
```

The function must display the number of documents removed.

TASK 2: MONGODB QUERIES, ARRAYS, TEXT SEARCH AND AGGREGATION [17 MARKS]

The CyberShield SOC analysts need to analyse the security events stored in MongoDB. Use the supplied **security_events.json** file.

Import the data into the **securityEvents** collection in the **cybershield** database.

Question 2.1 - Find Events by Severity [3 Marks]

Write a MongoDB query that retrieves all **High** and **Critical** security events.

Display only:

```
eventId
timestamp
sourceIP
```

```
eventType
severity
```

The results must be sorted by timestamp in descending order.

Question 2.2 - Query an Array Field [3 Marks]

The SOC analysts want to identify security events associated with the **brute-force** tag.

Write a MongoDB query that retrieves all events whose tags array contains: "brute-force"

Display:

```
eventId
eventType
severity
tags
```

Question 2.3 - Query Embedded Documents [3 Marks]

The `responseActions` field contains an array of embedded documents.

Retrieve all security events where at least one response action has:

status = "Pending"

Display:

```
eventId
eventType
severity
responseActions
```

Question 2.4 - Aggregation: Events by Severity [4 Marks]

The SOC manager wants a summary of the number of security events recorded at each severity level. Write an **aggregation pipeline** that produces output containing:

```
severity
numberOfEvents
```

For example:

Severity	Number of events
Critical	12
High	25
Medium	18
Low	7

The results must be sorted from the highest number of events to the lowest.

Question 2.5 - Text Search [4 Marks]

The SOC analysts frequently need to search security event descriptions for specific terms. Create an appropriate text index and use MongoDB text search to find security events containing the word:

authentication

The query must display:

```
eventId
eventType
severity
description
```

TASK 3: JAVA/MONGODB SOC APPLICATION [15 MARKS]

CyberShield wants a simple Java application that can be used by SOC analysts to interact with the security event database.

Develop a Java application that connects to the cybershield MongoDB database.

The application must provide a graphical user interface.

Question 3.1 - MongoDB Connection [3 Marks]

Configure the Java application to connect successfully to the MongoDB database, access the collections. The application must provide an appropriate response if the connection cannot be established.

Question 3.2 - SOC Analyst GUI [2 Marks]

Develop a Java Swing / JavaFX GUI that provides appropriate controls for the SOC analyst to interact with the database.

Your interface should contain, at minimum, controls that allow the analyst to:

- search security events;
- update an event; and
- generate a security-event summary.

The exact design of the interface is left to you.

Question 3.3 - Search Security Events [4 Marks]

The SOC analyst must be able to enter a source IP address and click "Search Events".

The application must:

1. obtain the source IP entered by the analyst;
2. query MongoDB;
3. retrieve matching security events; and
4. display the results in the GUI.

If no matching events are found, the application must display an appropriate message.

Question 3.4 - Update Security Event [3 Marks]

The SOC analyst must be able to enter:

- an event ID; and
- a new severity.

When the analyst clicks, "Update Severity", the application must update the corresponding MongoDB document.

The application must display an appropriate message indicating whether the update was successful.

Question 3.5 - Security Summary [3 Marks]

Add a button "Security Summary", which, when clicked, the application must execute an aggregation pipeline and display the number of security events for each severity level.

For example:

SECURITY EVENT SUMMARY	
Severity	**Number of events**
Critical	12
High	25
Medium	18
Low	7

The values displayed must be obtained from MongoDB and must not be hard-coded.

MARK DISTRIBUTION

Task	Question	Description	Marks
Task 1	1.1	Database and collection	3
	1.2	Insert security events function	4
	1.3	Find events by IP function	4
	1.4	Update severity function	4
	1.5	Remove low-severity events function	3
		Task 1 Total	**18**
Task 2	2.1	Find events by severity	3
	2.2	Array query	3
	2.3	Embedded-document query	3
	2.4	Aggregation by severity	4
	2.5	Text search	4
		Task 2 Total	**17**
Task 3	3.1	MongoDB connection	3
	3.2	GUI	2
	3.3	Search security events	4
	3.4	Update security event	3
	3.5	Security summary	3
		Task 3 Total	**15**
		GRAND TOTAL	**50**

IMPORTANT REQUIREMENTS

1. MongoDB Shell Functions

All functions developed in Task 1 must be stored in:

`.mongoshrc.js`

Students must demonstrate that the functions execute successfully from the MongoDB shell.

2. No hard-coded results

For Tasks 2 and 3, students must **retrieve results from MongoDB**. Hard-coded output will not receive marks.

3. Java Application

The Java application must actually communicate with MongoDB. A GUI that only displays pre-entered or hard-coded information will not receive the relevant marks.